

BAB 5

ALGORITMA KLASIFIKASI DECISION TREE

5.1 TUJUAN

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

1. Memahami konsep algoritma klasifikasi *decision tree*.
2. Menentukan kriteria percabangan yang tepat pada *decision tree*.
3. Mengimplementasikan algoritma *decision tree* dengan berbagai kriteria percabangan.

5.2 ALAT-ALAT YANG DIGUNAKAN

Pada percobaan ini, akan digunakan berbagai alat dan bahan yang mendukung kegiatan praktikum. Alat-alat tersebut meliputi:

- Daftar alat utama: OS Windows/Linux/macOS, Anaconda, Google Colab
- Pustaka: pandas, numpy, xgboost

5.3 DASAR TEORI

5.3.1 DECISION TREE

Decision tree merupakan sebuah algoritma klasifikasi yang memanfaatkan struktur data *tree* untuk menentukan kelas dari suatu data. Terdapat tiga jenis *node* pada *decision tree*:

1. *Root node*, merupakan *node* yang tidak memiliki *edge* masukan dan memiliki nol atau lebih *edge* keluaran.
2. *Internal node*, memiliki tepat satu *edge* masukan dan memiliki dua atau lebih *edge* keluaran. Pada *decision tree*, *internal node* berfungsi menampung variabel yang mengalami percabangan.
3. *Leaf* atau terminal *node*, mempunyai tepat satu *edge* masukan dan tidak mempunyai *edge* keluaran. Pada *decision tree*, *leaf* berfungsi untuk menyimpan kelas suatu data.

Tahapan pertama dalam pembentukan *decision tree* adalah menentukan variabel apa yang mengalami percabangan. Idealnya, setelah percabangan terbentuk distribusi kelas yang homogen. Terdapat beberapa kriteria yang dapat digunakan untuk pembentukan cabang:

1. Gini Index

$$GINI(t) = 1 - \sum_j [p(j|t)]^2$$
$$GINI_{split} = \sum_{i=1}^k \frac{n_i}{n} GINI(i)$$

Keterangan:

t : sebuah *node* pada *tree*

$p(j|t)$: frekuensi kelas j pada *node* t

n_i : Banyaknya data pada *child* i

n : Banyaknya data pada *node* p

2. Information Gain

$$GAIN_{split} = Entropy(p) - \left(\sum_{i=1}^k \frac{n_i}{n} Entropy(i) \right)$$

$$Entropy(t) = - \sum_j p(j|t) \log_2 p(j|t)$$

3. Gain Ratio

$$GainRATIO_{split} = \frac{GAIN_{split}}{SplitINFO}$$

$$SplitINFO = - \sum_{i=1}^k \frac{n_i}{n} \log_2 \frac{n_i}{n}$$

Pembentukan *tree* pada *decision tree* dilakukan menggunakan alur umum sebagai berikut:

1. Hitung kriteria pembentukan cabang (pilih salah satu) untuk setiap fitur pada data.
2. Lakukan percabangan menggunakan nilai kriteria tertinggi (Information Gain & Gain Ratio) atau kriteria terendah (Gini Index).
3. Ulangi langkah 1-2 untuk membentuk cabang menggunakan fitur yang tersisa.

5.3.2 XGBOOST

XGBoost (Extreme Gradient Boosting) merupakan pustaka *gradient-boosted decision trees*, yaitu implementasi *decision tree* yang dioptimalkan dari algoritma *boosting* yang menggunakan gradient descent, yang dirancang untuk efisiensi komputasi, kinerja prediktif yang tinggi, dan kemampuannya yang dapat menangani *dataset* besar. Algoritma ini sering digunakan dalam berbagai kompetisi *machine learning* karena kemampuannya dalam menangani berbagai jenis data dan menghasilkan model dengan akurasi tinggi. XGBoost dikembangkan oleh Tianqi Chen dari University of Washington. Kelebihannya antara lain dapat melakukan komputasi secara paralel dan terdistribusi, merupakan algoritma *cache-aware prefetching* untuk mengurangi *runtime* pada *dataset* besar, memiliki *regularization* untuk mencegah *overfitting*, dan dapat menangani masalah *missing value*. Namun, kelemahan XGBoost yaitu memiliki banyak *hyperparameter* yang perlu disetel dengan tepat, dan konsumsi memori untuk *dataset* yang sangat besar.

XGBoost dapat digunakan untuk klasifikasi dan regresi. Identya sederhananya adalah mengombinasikan (*ensemble*) *weak learner tree* dengan *strong learner* dengan cara menambahkan residual, mirip seperti Random Forest. Seperti teknik *boosting* lainnya, *gradient boosting* dimulai dengan *weak learner* (*weak classifier*) untuk membuat prediksi. Pohon keputusan pertama dalam *gradient boosting* disebut dengan *base learner*. Selanjutnya, pohon-pohon baru dibuat dengan cara penambahan (*additive*) berdasarkan kesalahan-kesalahan dari *base learner*. Algoritma kemudian menghitung residual dari setiap prediksi pohon untuk menentukan seberapa jauh prediksi model dari kenyataan. Residual adalah perbedaan antara prediksi model dan nilai aktual. Residual kemudian digabungkan untuk menilai model dengan fungsi *loss*.

5.3.3 ALGORITMA XGBOOST

Algorithm XGBoost($I, L, K, \eta, \lambda, \gamma$)

Input:

I : a training set $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ with feature dimension m

$L(y, \hat{y})$: a differentiable loss function

K : number of boosting iterations

η : the learning rate

λ : regularization coefficient

γ : minimum loss reduction required for a split

- 1: Initialize the model with a constant value:

$$F_0(\mathbf{x}) = \underset{\gamma}{\operatorname{argmin}} \sum_{i=1}^n L(y_i, \gamma)$$

- 2: **for** $k = 1$ to K **do**:
 3: /* Compute the sum of gradients and Hessians of all the samples */
 4: $G \leftarrow \sum_{i \in I} \frac{\partial L(y_i, F_{k-1}(\mathbf{x}_i))}{\partial F_{k-1}(\mathbf{x}_i)}$
 5: $H \leftarrow \sum_{i \in I} \frac{\partial^2 L(y_i, F_{k-1}(\mathbf{x}_i))}{\partial F_{k-1}(\mathbf{x}_i)^2}$
 6: $T \leftarrow \text{Build-Tree}(I, G, H)$
 7: Let the terminal regions of T be R_j for $j = 1, \dots, J_k$
 8: For $j = 1, \dots, J_k$ compute:

$$w_j = -\frac{G_j}{H_j + \lambda}$$

where G_j and H_j are the sum of gradients and Hessians at leaf node j .

- 9: Update the model:

$$F_k(\mathbf{x}) = F_{k-1}(\mathbf{x}) + \eta \sum_{j=1}^{J_k} w_j \mathbb{1}\{\mathbf{x} \in R_j\}$$

- 10: Output the final ensemble $F_K(\mathbf{x})$

Algoritma ini menggunakan fungsi yang disebut Build-Tree untuk membangun pohon regresi berikutnya dalam ansambel

Function Build-Tree(I, G, H, F_{k-1})

Input:

I : the training set
 G : sum of the gradients of the samples in I
 H : sum of the Hessians of the samples in I
 F_{k-1} : the ensemble from the previous iteration

- 1: Create a *root* node for the tree and store the tuple (I, G, H) in it
 2: $(\text{gain}, I_L, G_L, H_L, I_R, G_R, H_R) \leftarrow \text{Find-Best-Split}(I, G, H, F_{k-1})$
 3: **if** $\frac{1}{2} \cdot \text{gain} < \gamma$ **then**
 4: **return** *root*
 5: **else**
 6: $\text{subtree} \leftarrow \text{Build-Tree}(I_L, G_L, H_L, F_{k-1})$
 7: Add *subtree* to the left branch of *root*
 8: $\text{subtree} \leftarrow \text{Build-Tree}(I_R, G_R, H_R, F_{k-1})$
 9: Add *subtree* to the right branch of *root*
 10: **return** *root*
-

Fungsi ini kemudian menggunakan fungsi pembantu yaitu Find-Best-Split, yang mengevaluasi setiap kemungkinan pembagian pada simpul yang diberikan dan mengembalikan pembagian/pemisahan dengan *gain* tertinggi (disebut algoritma *exact greedy* pada paper asli XGBoost). Pemisahan terbaik dikembalikan sebagai sebuah *tuple* yang berisi himpunan bagian dari sampel-sampel yang menuju ke node anak kiri dan kanan, serta jumlah gradien dan Hessiannya. *Pseudocode* dari fungsi ini ditunjukkan di bawah ini:

Function Find-Best-Split(I, G, H, F_{k-1})

Input:

I : the set of samples at the current node
 G : sum of the gradients of the samples in I
 H : sum of the Hessians of the samples in I
 F_{k-1} : the ensemble from the previous iteration

```

1:  $gain \leftarrow 0$ 
2:  $best\_split \leftarrow None$ 
3: /* Consider all the features */
4: for  $j = 1$  to  $m$  do
5:    $I_L \leftarrow \emptyset, G_L \leftarrow 0, H_L \leftarrow 0$ 
6:   /* Compute the gain from each possible split point of feature  $j$  */
7:   Sort the samples in  $I = \{(\mathbf{x}_i, y_i)\}$  by the values of  $x_{ij}$ 
8:   for  $i \in I_{sorted}$  do
9:      $g_i \leftarrow \frac{\partial L(y_i, F_{k-1}(\mathbf{x}_i))}{\partial F_{k-1}(\mathbf{x}_i)}$ 
10:     $h_i \leftarrow \frac{\partial^2 L(y_i, F_{k-1}(\mathbf{x}_i))}{\partial F_{k-1}(\mathbf{x}_i)^2}$ 
11:     $I_L \leftarrow I_L \cup \{i\}, G_L \leftarrow G_L + g_i, H_L \leftarrow H_L + h_i$ 
12:     $I_R \leftarrow I - I_L, G_R \leftarrow G - G_L, H_R \leftarrow H - H_L$ 
13:     $\Delta \leftarrow \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda}$ 
14:    if  $\Delta > gain$  then
15:       $gain \leftarrow \Delta$ 
16:       $best\_split \leftarrow (I_L, G_L, H_L, I_R, G_R, H_R)$ 
17: return  $gain, best\_split$ 

```

5.3.4 RINGKASAN

Decision Tree adalah algoritma machine learning yang digunakan untuk tugas klasifikasi dan regresi. Algoritma ini membangun model berbentuk pohon keputusan dengan membagi data secara rekursif berdasarkan fitur-fitur yang paling informatif. Setiap *node* dalam pohon merepresentasikan suatu keputusan berdasarkan nilai fitur, sedangkan *leaf node* merepresentasikan hasil prediksi. Kelebihan Decision Tree antara lain kemudahan interpretasi, kemampuan menangani data numerik dan kategorikal, serta tidak memerlukan normalisasi data. Namun, Decision Tree rentan terhadap *overfitting*, terutama pada *dataset* yang besar dan kompleks, serta kurang stabil karena perubahan kecil pada data dapat menghasilkan pohon yang sangat berbeda.

XGBoost (Extreme Gradient Boosting) adalah algoritma yang lebih canggih dan dioptimalkan dari konsep Gradient Boosting Machines (GBM). XGBoost membangun model secara iteratif dengan menggabungkan beberapa pohon keputusan yang lemah (*weak learners*) untuk membentuk model yang kuat. Setiap pohon baru dibangun untuk memperbaiki kesalahan prediksi dari pohon sebelumnya. Kelebihan XGBoost meliputi kecepatan dan efisiensi komputasi, akurasi yang tinggi, fleksibilitas dalam menangani berbagai fungsi tujuan, serta kemampuan untuk menangani data hilang dan regularisasi. Namun, XGBoost memiliki kelemahan seperti kompleksitas dalam *tuning hyperparameter*, waktu pelatihan yang lama untuk *dataset* besar, dan risiko *overfitting* jika tidak diatur dengan benar.

Contoh algoritma serupa adalah RandomForest, yang juga berbasis pohon keputusan. RandomForest membangun banyak pohon keputusan secara paralel (*ensemble*) dan menggabungkan hasilnya melalui voting (untuk klasifikasi) atau *averaging* (untuk regresi). Kelebihan RandomForest adalah kemampuannya mengurangi *overfitting* dibandingkan Decision Tree, stabilitas yang lebih baik, dan kemudahan penggunaan. Namun, RandomForest kurang interpretatif dibandingkan Decision Tree dan memerlukan lebih banyak sumber daya komputasi.

Kesimpulannya, Decision Tree cocok untuk masalah sederhana dan interpretabilitas tinggi,

sementara XGBoost dan RandomForest lebih cocok untuk masalah kompleks dengan akurasi tinggi, meskipun memerlukan lebih banyak sumber daya dan *tuning*.

5.4 PROSEDUR PERCOBAAN

5.4.1 IMPOR DATA

Praktikum kali ini menggunakan *dataset* [Car Evaluation Dataset](https://archive.ics.uci.edu/ml/datasets/Car+Evaluation)¹ dari UCI Machine Learning Repository. *Dataset* ini telah digunakan pada praktikum sebelumnya. Detail fitur dapat Anda pelajari pada link yang tersedia.

Unduh *dataset* yang akan digunakan pada praktikum kali ini. Anda dapat menggunakan aplikasi *wget* untuk mengunduh *dataset* dan menyimpannya dalam Google Colab. Jalankan *cell di bawah* ini untuk mengunduh *dataset*.

```
1 ! wget https://github.com/adikara-ub/praktikum-ai-lanjut/raw/main/dataset/car_sample.csv
```

Setelah *dataset* berhasil diunduh, langkah berikutnya adalah membaca *dataset* dengan memanfaatkan fungsi *readcsv* dari library *pandas*. Lakukan pembacaan berkas *csv* ke dalam *DataFrame* dengan nama data menggunakan fungsi *readcsv*. Jangan lupa untuk melakukan import library *pandas* terlebih dahulu

```
1 import pandas as pd
2 import numpy as np
3 data = pd.read_csv('car_sample.csv')
```

Cek isi *dataset* Anda dengan menggunakan perintah **head()**

```
1 data.head()
```

5.4.2 MEMBAGI DATA MENJADI DATA LATIH DAN DATA UJI

Metode pembelajaran mesin memerlukan dua jenis data:

1. Data latih: Digunakan untuk proses *training* metode klasifikasi
2. Data uji: Digunakan untuk proses evaluasi metode klasifikasi

Data uji dan data latih perlu dibuat terpisah (*mutually exclusive*) agar hasil evaluasi lebih akurat. Data uji dan data latih dapat dibuat dengan cara membagi *dataset* dengan rasio tertentu, misalnya 80% data latih dan 20% data uji.

Library Scikit-learn memiliki fungsi [train_test_split](#) pada modul *model_selection* untuk membagi *dataset* menjadi data latih dan data uji. Bagilah *dataset* anda menjadi dua, yaitu **data_latih** dan **data_uji**. Agar pengacakan data dilakukan secara konstan, parameter **random_state** diisi dengan nilai integer tertentu, pada praktikum ini disetel 101. Kemudian, nilai indeks pada data latih dan data uji diatur ulang agar berurutan nilainya.

```
1 from sklearn.model_selection import train_test_split
2 data_latih, data_uji = train_test_split(data, test_size=0.2, random_state=101)
3 data_latih.reset_index(drop=True)
```

¹ <https://archive.ics.uci.edu/ml/datasets/Car+Evaluation>


```
4 data_uji.reset_index(drop=True)
```

Tampilkan banyaknya data pada **data_latih** dan **data_uji**. Seharusnya **data_latih** terdiri dari 208 data, dan **data_uji** terdiri dari 52 data.

```
1 print(data_uji.shape[0])
2 print(data_latih.shape[0])
```

5.4.3 MENGHITUNG GINI

Nilai Gini merupakan salah satu kriteria penentu variabel apa yang akan digunakan untuk membentuk cabang pada *decision tree*. Variabel dengan nilai Gini terendah akan digunakan sebagai pembentukan cabang

Buatlah fungsi bernama **hitung_gini** yang berfungsi menghitung nilai Gini dari suatu nilai pada sebuah variabel.

```
1 def hitung_gini(kolom_kelas):
2     elemen,banyak = np.unique(kolom_kelas,return_counts = True)
3     nilai_gini = 1 - np.sum([(banyak[i]/np.sum(banyak))**2 for i in range(len(elemen))])
4     return nilai_gini
```

Fungsi **hitung_gini** menerima nilai kelas pada variabel tertentu dengan nilai tertentu. Nilai dan frekuensi dari masing-masing kelas dihitung menggunakan fungsi [np.unique](#) dari library numpy, kemudian dilakukan kalkulasi nilai GINI.

Nilai GINI total dari sebuah variabel dihitung menggunakan fungsi **gini_split**.

```
1 def gini_split(data,nama_fitur_split,nama_fitur_kelas):
2     nilai,banyak= np.unique(data[nama_fitur_split],return_counts=True)
3     gini_split = np.sum([(banyak[i] / np.sum(banyak)) * hitung_gini(data.where(
4         data[nama_fitur_split]==nilai[i]).dropna()[nama_fitur_kelas]) for i in
5         range(len(nilai))])
6     return gini_split
```

Fungsi **gini_split** menerima input berupa data, nama fitur yang akan dilakukan percabangan, serta nama fitur yang mengandung kelas data. Tahapan pertama dari fungsi ini adalah mendapatkan setiap elemen dari variabel yang akan dilakukan percabangan beserta frekuensinya. Selanjutnya dilakukan perhitungan nilai GINIsplit sesuai persamaan yang diberikan, dengan menghitung terlebih dahulu nilai GINI pada masing-masing elemennya.

Ujilah fungsi **gini_split** menggunakan **data_latih** pada variabel **buying** dan variabel kelas bernama **class**.

```
1 gini_split(data_latih,"buying","class")
```

5.4.4 PEMBENTUKAN POHON

Pembentukan pohon dilakukan secara rekursif. Seperti metode rekursif pada umumnya, perlu ditentukan kondisi berhenti terlebih dahulu. Kondisi berhenti pada pembentukan pohon adalah:

1. Jika kelas pada data hanya ada satu, kembalikan kelas tersebut
2. Jika fitur data = 0 (tidak ada fitur yang tersisa), kembalikan kelas dari parent
3. Jika data kosong (tidak ada data), kembalikan kelas dengan frekuensi terbanyak

Selain kondisi berhenti tersebut, dilakukan pembentukan pohon secara rekursif menggunakan fungsi

buat_tree.

```

1 def buat_tree(data,data_awal, daftar_fitur, nama_fitur_kelas,kelas_parent_node=None):
2     #jika hanya ada satu kelas pada data
3     if len(np.unique(data[nama_fitur_kelas])) <= 1:
4         return np.unique(data[nama_fitur_kelas])[0]
5     #jika data kosong
6     elif len(data)==0:
7         return np.unique(data_awal[nama_fitur_kelas])
8         [np.argmax(np.unique(data_awal[nama_fitur_kelas],return_counts=True)[1])]
9     #jika tidak ada fitur yang terisa
10    elif len(daftar_fitur) ==0:
11        return kelas_parent_node
12    else:
13        kelas_parent_node = np.unique(data[nama_fitur_kelas])
14        [np.argmax(np.unique(data[nama_fitur_kelas],return_counts=True)[1])]
15        nilai_split = [gini_split(data,fitur,nama_fitur_kelas) for fitur in daftar_fitur]
16        index_fitur_terbaik = np.argmin(nilai_split)
17        fitur_terbaik = daftar_fitur[index_fitur_terbaik]
18        tree = {fitur_terbaik:{}}
19        daftar_fitur = [i for i in daftar_fitur if i != fitur_terbaik]
20        for nilai in np.unique(data[fitur_terbaik]):
21            sub_data = data.where(data[fitur_terbaik] == nilai).dropna()
22            subtree = buat_tree(sub_data,data_awal,daftar_fitur,nama_fitur_kelas,
23                               kelas_parent_node)
24            tree[fitur_terbaik][nilai]=subtree
25    return(tree)

```

Fungsi **buat_tree** memiliki paramter input berupa:

1. data: merupakan data yang membentuk *tree*. Seiring dengan bertambahnya kedalaman *tree*, maka akan ada fitur yang dihapus pada data
2. data_awal: merupakan data latih awal yang tidak mengalami penghapusan fitur.
3. daftar_fitur: merupakan daftar fitur-fitur yang digunakan dalam pembentukan *tree*. Seiring dengan bertambahnya kedalaman *tree*, maka akan ada fitur yang dihapus dari daftar_fitur.
4. nama_fitur_kelas: merupakan nama fitur pada data yang mengandung kelas data.
5. Kelas_parent_node: menunjukkan parent *node* dari suatu *node*

Variabel **kelas_parent_node** berisi kelas dengan frekuensi terbesar pada suatu *node*. Variabel ini berguna ketika kondisi berhenti nomor 2 terpenuhi (tidak ada fitur yang tersisa). Variabel **nilai_split** merupakan sebuah *list* yang berisi nilai kriteria percabangan (pada kasus ini, nilai GINIsplit) pada masing-masing fitur. **index_fitur_terbaik** dan **fitur_terbaik** berisi indeks dan nama fitur dengan nilai kriteria percabangan terendah. Selanjutnya, variabel **tree** berisi *decision tree* yang menggunakan struktur data *dictionary* dengan key berupa nama fitur dengan nilai kriteria percabangan tertinggi (**fitur_terbaik**). Selanjutnya, fitur dengan nilai kriteria percabangan tertinggi dihapus dari daftar fitur yang akan digunakan untuk membuat *decision tree* (**daftar_fitur**). Setelah itu, dilakukan looping pada masing-masing nilai di fitur **fitur_terbaik** untuk membentuk subtree dengan cara memanggil fungsi **buat_tree** secara rekursif. Ujilah fungsi **buat_tree** menggunakan data latih yang tersedia.

```

1 tree = buat_tree(data_latih,data_latih,data_latih.columns[:-1], 'class')

```

Tampilkan *tree* yang terbentuk. unakan library **pprint** untuk menampilkan *dictionary* secara teratur.

```

1 from pprint import pprint
2 pprint(tree)

```

5.4.5 PROSES PREDIKSI DECISION TREE

Proses prediksi kelas pada data uji dilakukan dengan melakukan *tree* traversal sampai menemui *leaf*.

```
1 def prediksi(data_uji,tree):
2     for key in list(data_uji.keys()):
3         if key in list(tree.keys()):
4             try:
5                 hasil = tree[key][data_uji[key]]
6             except:
7                 return 1
8             hasil = tree[key][data_uji[key]]
9             if isinstance(hasil,dict):
10                return prediksi(data_uji,hasil)
11            else:
12                return hasil
```

Fungsi **prediksi** menerima input berupa data uji dan *tree* hasil *training*, keduanya dalam struktur data *dictionary*. Tahapan pertama dalam prediksi adalah memperoleh semua key dari data uji. Kemudian, setiap key pada data uji akan dicek apakah menjadi root *node* pada *tree*. Jika benar, maka variabel **hasil** akan berisi *child* dari root pada *tree* dengan nilai yang sama dengan data uji. Jika variabel **hasil** merupakan sebuah *dictionary*, maka dilakukan pemanggilan fungsi **prediksi** secara rekursif. Jika tidak, maka variabel **hasil** merupakan *leafnode* yang berisi kelas dari data uji. Perhatikan bahwa terdapat penggunaan **try except** pada pencarian hasil. Hal ini dilakukan untuk mengantisipasi apabila ada nilai pada data uji yang tidak terdapat pada *decision tree*. Jika ini terjadi, maka kelas data uji diberi nilai 1.

5.4.6 PROSES PENGUJIAN DECISION TREE

Lakukan pengujian menggunakan data uji. Kelas pada data uji perlu dihapus dan data uji perlu diubah menjadi *dictionary*.

```
1 data_uji_dict = data_uji.iloc[:, :-1].to_dict(orient = "records")
```

Lakukan pengujian terhadap keseluruhan data uji menggunakan looping.

```
1 hasil_prediksi_total = []
2 for i in range(len(data_uji_dict)):
3     hasil_prediksi = prediksi(data_uji_dict[i],tree)
4     hasil_prediksi_total.append(hasil_prediksi)
```

Bandingkan hasil prediksi dengan label sebenarnya. Hitunglah banyaknya data uji yang memiliki kelas prediksi sama dengan kelas sebenarnya.

```
1 print("Total prediksi benar: ",sum(hasil_prediksi_total==data_uji['class']))
2 print("Total prediksi salah: ",sum(hasil_prediksi_total!=data_uji['class']))
3 print("Total data: ", len(data_uji))
```

5.4.7 PROSES PREDIKSI DENGAN XGBOOST

Dalam percobaan ini kita tidak membuat XGBoost dari awal namun menggunakan pustaka yang sudah ada. Lakukan instalasi dengan sintaksis berikut.

```
pip install xgboost
```

atau dengan conda:

```
conda install -c conda-forge py-xgboost
```


Apabila ada masalah dalam instalasi untuk mendeteksi CPU atau GPU, gunakan perintah berikut:
CPU only

```
conda install -c conda-forge py-xgboost-cpu
```

Use NVIDIA GPU

```
conda install -c conda-forge py-xgboost-gpu
```

Impor pustaka yang diperlukan:

```
1 from xgboost import XGBClassifier
2 from sklearn.model_selection import train_test_split
3
```

Dengan *dataset* sebelumnya kita coba langsung gunakan atau muat *file* CSV ke *DataFrame* *df*. Buat model berdasarkan data dan *classifier* *XGBClassifier*:

```
4 from sklearn.preprocessing import LabelEncoder
5 label_encoder = LabelEncoder()
6 df_new = df.apply(label_encoder.fit_transform)
7
8 X_train, X_test, y_train, y_test = train_test_split(df_new.loc[:,
    'buying':'safety'], df_new['class'], test_size=.2)
9 # create model instance
10 bst = XGBClassifier(n_estimators=2, max_depth=2, learning_rate=1,
    objective='binary:logistic')
```

Untuk melakukan pelatihan panggil metode **fit()** dan berikan data latih **X_train** dan target **y_train**:

```
# fit model
bst.fit(X_train, y_train)
```

Untuk memprediksi data pada **X_test**:

```
# make predictions
y_pred = bst.predict(X_test)
```

Tampilkan hasil yang masih di-*encoding* (data numerik 0, 1, dst):

```
print(y_pred)
```

Tampilkan hasil dengan nama *class* asli (data kategori) sebelum di-*encoding*:

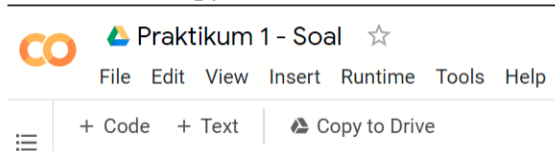
```
y_label = label_encoder.inverse_transform(y_pred)
print(y_label)
```

5.5 TUGAS

Pada tugas kali ini Anda diminta memodifikasi metode pembentukan *tree* yang telah Anda agar metode tersebut menggunakan Information Gain sebagai dasar percabangan. Lengkapilah kerangka *source code* pada *notebook* yang tersedia dan jawablah pertanyaan yang ada.

Petunjuk pengerjaan soal:

1. Buka tautan berikut, pastikan Anda *login* menggunakan **akun Student UB**.
<https://s.ub.ac.id/nbkal05>
2. Klik tombol **Copy to Drive**



3. Beri nama file **Praktikum 5 - Nama – NIM.ipynb**
4. Isilah *cell* yang kosong.
5. Unduh file ***.ipynb** dengan cara klik **File > Download .ipynb**.
6. Kumpulkan file ***.ipynb** ke asisten.

5.6 KESIMPULAN

Setelah melakukan semua percobaan ini maka mahasiswa diwajibkan membuat kesimpulan.

Kesimpulan dari percobaan ini merupakan bagian penting untuk merangkum temuan utama dan relevansinya dengan tujuan percobaan. Mahasiswa harus mampu menyimpulkan:

- Apakah tujuan percobaan telah tercapai?
- Hubungan antara hasil percobaan dan dasar teori.
- Faktor-faktor yang memengaruhi hasil percobaan, termasuk kemungkinan kesalahan yang terjadi.

Kesimpulan harus disusun secara singkat, jelas, dan berdasarkan data yang diperoleh.